EASTERN INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF SOFTWARE ENGINEERING

EIU
TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG
EASTERN
INTERNATIONAL
UNIVERSITY

MOBILE APPLICATION DEVELOPMENT REPORT

BABYNUTRI

Student (s)

Nguyễn Đức Anh – 2031200076

Nguyễn Minh Nguyên - 2231209001

Nguyễn Hoàng Anh Khoa - 2231200100

Lecturer

Mr. Trần Văn Tài

Ho Chi Minh, August 2026

ABSTRACT

This report describes the development of BABYNUTI, an infant nutrition and growth tracking application built for parents and caregivers. The app allows users to manage child profiles, discover weaning recipes filtered by age and allergies, plan weekly meals, track growth measurements through interactive charts, and access expert nutrition articles. The system is built using Express.js for the backend, MySQL for the database, and React Native with TypeScript for the Android app. This combination provides a smooth and easy-to-use experience for Android users. Overall, the app focuses on personalized nutrition management, intuitive growth monitoring, and structured meal planning to help parents support their child's healthy development.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

No.	Term	Meaning
1	ACID	Atomicity, Consistency, Isolation, Durability
2	API	Application Programming Interface
3	BMI	Body Mass Index
4	CRUD	Create, Read, Update, Delete
5	ERD	Entity Relationship Diagram
6	HTTP	HyperText Transfer Protocol
7	i18n	Internationalization
8	IDE	Integrated Development Environment
9	JSON	JavaScript Object Notation
10	JWT	JSON Web Token
11	OAuth	Open Authorization
12	RDBMS	Relational Database Management System
13	REST	Representational State Transfer
14	SDK	Software Development Kit
15	UI	User Interface
16	UX	User Experience
17	WHO	World Health Organization

CHAPTER 1. OVERVIEW

1.1. Introduction

Proper nutrition during early childhood is crucial for healthy physical growth and cognitive development. The BABYNUTI application serves as a comprehensive mobile companion for parents to manage their infant's weaning journey. It allows users to create personalized child profiles, discover age-appropriate and allergy-safe recipes, plan weekly meals, and track physical growth over time — all from their Android device.

1.2. Project Objectives

The specific objectives of the BABYNUTI mobile application project are as follows:

1. To develop a native Android mobile application using React Native (v0.86.0) and TypeScript, targeting Android devices and following a component-based, single-codebase architecture that enables future iOS expansion.
2. To implement a secure, dual-mode authentication system supporting both conventional email/password login and Google Sign-In via Firebase Authentication, with JWT-based session management and persistent token storage using AsyncStorage.
3. To build a comprehensive child profile management module that allows parents to create, update, and delete child profiles containing essential attributes such as the child's name, date of birth, gender, avatar, known allergies, and nutritional goals, and to support shared caregiver access through an invitation code system.
4. To develop a recipe browsing, search, and filtering system that enables parents to discover weaning-appropriate recipes filtered by the child's age in months, meal type (Breakfast, Lunch, Dinner, Snack), and dietary restrictions or allergy profiles, including the ability to save, rate, and comment on individual recipes.
5. To implement a weekly meal planning and scheduling module that allows parents to organize selected recipes into a structured weekly meal plan for a specific child, and to visualize the resulting schedule in a calendar-style meal scheduler interface.
6. To develop a growth tracking module with the ability to record and persist height and weight measurements over time, displayed as an interactive time-series chart, supporting configurable measurement units (centimeters/inches, kilograms/pounds).

7. To integrate multi-language support and user-configurable application settings including interface language switching via i18next, dark/light/system theme selection persisted through Redux and AsyncStorage, and measurement unit preferences managed through a dedicated backend settings endpoint.
8. To design and implement a scalable three-tier RESTful API backend using Node.js and Express.js, with structured route–controller–service layering, JWT authentication middleware, Firebase Admin SDK token verification, and MySQL as the relational data store, providing a robust foundation for future feature expansion.

CHAPTER 2. TECHNOLOGY

2.1. React Native

React Native is an open-source mobile application framework developed by Meta that enables developers to build native Android and iOS applications using JavaScript and the React programming model. Unlike hybrid approaches, React Native compiles to genuine native UI components, resulting in a near-native performance and user experience.

Advantages of React Native:

- Enables cross-platform development from a single codebase, significantly reducing development time and cost.
- Renders genuine native UI components rather than WebView wrappers, delivering a smooth and responsive user experience.
- Supported by a large and active community with an extensive ecosystem of third-party libraries.
- Provides fast refresh functionality, allowing developers to see changes in real time without a full rebuild.

Disadvantages of React Native:

- Performance may be marginally lower than fully native development for complex graphics or heavy computations.
- Integrating native device features requires additional platform-specific configuration, increasing setup complexity.
- Framework updates can occasionally introduce breaking changes that require dependency migration.

Why we use React Native: BABYNUTI is a mobile application targeting Android users, and React Native allows the entire team to build and maintain the application using a shared TypeScript codebase. Its extensive library ecosystem provides all the native integrations required by the project, including authentication, navigation, and data visualization.

2.2. TypeScript

TypeScript is an open-source programming language developed by Microsoft that extends JavaScript by adding static type definitions. It compiles to plain JavaScript and integrates seamlessly with existing JavaScript tools and frameworks.

Advantages of TypeScript:

- Detects type-related errors at compile time, significantly reducing runtime bugs in production.
- Provides rich IDE support including intelligent code completion, inline documentation, and safe refactoring.
- Improves code readability and maintainability, particularly in large projects with multiple contributors.
- Enables precise definition of data structures shared across modules, ensuring consistency throughout the codebase.

Disadvantages of TypeScript:

- Introduces a compilation step and requires explicit type annotations, increasing initial development effort.
- Some third-party libraries have incomplete or missing type definitions, requiring manual workarounds.

Why we use TypeScript: BABYNUTI uses TypeScript across the entire frontend to enforce type safety between the API service layer, state management, and UI components. This reduces integration errors significantly in a multi-developer environment.

2.3. React Navigation

React Navigation is the standard navigation library for React Native applications. It provides a fully customizable, JavaScript-based navigation solution supporting native stack navigators, bottom tab navigators, drawer navigators, and modal presentations.

Advantages of React Navigation:

- Offers a declarative, component-based API for defining complex, multi-level navigation hierarchies.
- Provides full TypeScript support for typed navigation parameters passed between screens.
- Leverages the native platform's gesture system to deliver authentic navigation transitions.
- Highly extensible, supporting custom tab bars, headers, and animated transitions.

Disadvantages of React Navigation:

- Configuration of deeply nested navigators can become verbose and difficult to maintain.
- JavaScript-based animations may exhibit minor performance issues on low-end devices compared to fully native navigation.

Why we use React Navigation: BABYNUTI implements a two-level navigation hierarchy — a root stack navigator managing the authentication flow and all feature screens, and a bottom tab navigator providing access to the four main sections of the application. React Navigation's TypeScript integration ensures all screen-to-screen parameter passing is fully type-safe.

2.4. Redux Toolkit

Redux Toolkit is the officially recommended library for global state management in Redux-based JavaScript applications. It provides a set of utilities including createSlice and createAsyncThunk that simplify writing Redux logic while enforcing best practices and reducing boilerplate.

Advantages of Redux Toolkit:

- Eliminates traditional Redux boilerplate by combining reducers and action creators in a single, concise API.
- createAsyncThunk provides a structured pattern for managing asynchronous operations with built-in lifecycle states.
- Integrates with Redux DevTools, enabling transparent state inspection during development.
- Works seamlessly with TypeScript for fully typed state and dispatch operations.

Disadvantages of Redux Toolkit:

- Introduces additional configuration overhead that may be excessive for managing simple, local UI state.
- Requires developers to learn Redux concepts and patterns before being productive.

Why we use Redux Toolkit: BABYNUTI uses Redux Toolkit to manage application-wide state that must persist across screen navigation, including the authenticated user's identity, the active child profile, display theme, interface language, and measurement unit preferences.

2.5. Zustand

Zustand is a lightweight, unopinionated state management library for React and React Native. It provides a minimal API based on JavaScript closures and React hooks, requiring no context providers or reducer functions.

Advantages of Zustand:

- Extremely minimal API — an entire store is defined with a single function call, requiring no boilerplate.
- Does not require wrapping component trees in Provider components.
- Naturally supports optimistic updates and asynchronous actions without additional middleware.
- Significantly smaller bundle footprint compared to Redux.

Disadvantages of Zustand:

- Less suitable for complex, interdependent global state that benefits from Redux's structured action history.
- Lacks the built-in middleware and DevTools ecosystem that Redux provides.

Why we use Zustand: BABYNUTI uses Zustand alongside Redux Toolkit to manage feature-scoped state for recipe browsing, article listing, and bookmarked items. Its simple synchronous state access enables clean optimistic UI patterns, such as immediately removing an item from the displayed list before the API deletion call completes.

2.6. Axios

Axios is a promise-based HTTP client for JavaScript that supports request and response interceptors, automatic JSON transformation, configurable timeouts, and structured error handling. It is available for both browser and Node.js environments.

Advantages of Axios:

- Request and response interceptors allow authentication headers and error handling to be configured once and applied globally.
- Automatically serializes and deserializes JSON request and response bodies.
- Provides cleaner error objects with structured response, request, and message fields.
- Supports request timeout configuration to prevent indefinitely hanging calls.

Disadvantages of Axios:

- Adds an external dependency over the built-in Fetch API, increasing bundle size slightly.
- For simple requests, the native Fetch API may be equally sufficient without the additional overhead.

Why we use Axios: BABYNUTI configures an Axios instance with a request interceptor that automatically reads the user's authentication token from device storage and attaches it to every outgoing API request, ensuring all authenticated calls are handled consistently without repeating this logic in individual service functions.

2.7. Firebase Authentication

Firebase Authentication is a backend-as-a-service provided by Google that supports multiple sign-in methods including email/password, Google, and other OAuth providers. It is integrated into the React Native application via the @react-native-firebase/auth package and the @react-native-google-signin/google-signin SDK.

Advantages of Firebase Authentication:

- Provides a fully managed, secure OAuth 2.0 flow for Google Sign-In with minimal frontend configuration.
- Firebase ID tokens are cryptographically verifiable on the backend using the Firebase Admin SDK, requiring no additional secret exchange.
- Handles token lifecycle, session management, and token refresh automatically.

Disadvantages of Firebase Authentication:

- Requires platform-specific native configuration for Android, adding setup overhead for bare React Native projects.
- Introduces a dependency on Google's external infrastructure, which may raise data residency concerns for some deployments.

Why we use Firebase Authentication: BABYNUTI supports Google Sign-In as a login method. Firebase Authentication handles the secure OAuth flow on the client side, and the resulting identity token is verified on the backend using the Firebase Admin SDK before a custom application JWT is issued.

2.8. AsyncStorage

@react-native-async-storage/async-storage is the community-maintained asynchronous persistent key-value storage library for React Native. It is analogous to localStorage in web browsers but designed for the constraints of mobile device storage.

Advantages of AsyncStorage:

- Provides simple, promise-based persistence that survives application restarts and device reboots.
- Cross-platform compatible with both Android and iOS.
- Integrates naturally with Redux async thunks for restoring persisted state on app launch.

Disadvantages of AsyncStorage:

- Not suitable for storing large data structures or sensitive cryptographic material requiring hardware-level security.
- Asynchronous reads can introduce minor latency during application startup when rehydrating saved state.

Why we use AsyncStorage: BABYNUTI uses AsyncStorage to persist the user's authentication token across sessions and to save user preferences — including theme mode, language selection, and meal plan data — so that the application restores the correct state on every cold start.

2.9. i18next & react-i18next

i18next is a comprehensive internationalization framework for JavaScript. react-i18next is its React-specific binding, providing the useTranslation hook that allows any component to access the active locale's translation strings.

Advantages of i18next:

- Supports multiple languages with a clean, hook-based API that integrates naturally into functional components.
- Locale resources can be organized per feature or namespace for better maintainability.
- The active language can be changed at runtime, triggering an immediate full-UI re-render.

Disadvantages of i18next:

- Requires maintaining translation files for every supported language throughout the development lifecycle.
- Initial configuration and namespace structuring add upfront overhead to the project setup.

Why we use i18next: BABYNUTI is designed for a multilingual user base, and i18next provides the framework for switching the entire application interface between supported languages at runtime, with the active language preference persisted through the application's global state.

2.10. Node.js & Express.js

Node.js is an open-source, event-driven JavaScript runtime built on Chrome's V8 engine, designed for building scalable network applications. Express.js is a minimal and flexible web framework for Node.js that provides a concise API for building RESTful services.

Advantages of Node.js & Express.js:

- Node.js's non-blocking, asynchronous I/O model efficiently handles multiple concurrent API requests.
- Sharing JavaScript across both frontend and backend reduces context-switching for the development team.
- Express.js's rich middleware ecosystem simplifies common server-side tasks such as authentication, file uploads, and CORS handling.
- Express v5 natively propagates async/await errors, eliminating repetitive error-handling boilerplate in route handlers.

Disadvantages of Node.js & Express.js:

- Express.js is unopinionated by design, requiring the team to establish and enforce its own architectural structure.
- Node.js's single-threaded model is not optimal for CPU-bound tasks such as image processing.

Why we use Node.js & Express.js: BABYNUTI's backend API is organized in a clean route-controller-service layered architecture, with all endpoints registered under a unified base path. Express v5 is selected specifically for its native async error propagation, which simplifies controller code across all API modules.

2.11. MySQL

MySQL is a widely adopted open-source relational database management system (RDBMS) known for its reliability, ACID compliance, and strong support for complex relational queries. The backend communicates with MySQL using the mysql2 driver, which provides a high-performance Node.js interface with native Promise support.

Advantages of MySQL:

- Enforces referential integrity through foreign key constraints, ensuring consistency across all related entities.
- Well-suited to BABYNUTI's structured, highly relational data model involving users, children, recipes, meal plans, and growth records.
- The mysql2 connection pool manages concurrent database connections efficiently without manual lifecycle management.
- Widely supported with extensive documentation, tooling, and community resources.

Disadvantages of MySQL:

- Schema changes require deliberate migration management, particularly in a multi-developer environment.
- Writing raw SQL queries without an ORM requires careful input parameterization to prevent SQL injection vulnerabilities.

Why we use MySQL: BABYNUTI's data model comprises 28 interrelated tables with full referential integrity enforced throughout, ensuring that deleting a parent entity automatically removes all dependent child records. MySQL provides the relational structure and data consistency guarantees that this domain requires.

2.12. JSON Web Token (JWT) & bcrypt

JSON Web Tokens (JWT) are an open standard (RFC 7519) for securely transmitting authentication claims between parties as a digitally signed, self-contained token. bcrypt is a cryptographic hashing algorithm specifically designed for secure password storage, incorporating a salt and a configurable work factor.

Advantages of JWT & bcrypt:

- JWT tokens are stateless and self-contained, allowing the server to authenticate any request without maintaining session state.

- bcrypt's salt and work factor make brute-force and rainbow table attacks computationally infeasible.
- JWT verification is performed locally on the server without any external network calls.

Disadvantages of JWT & bcrypt:

- JWT tokens cannot be individually revoked before their expiry time without implementing an additional server-side blacklist.
- bcrypt is intentionally slow, which may impact throughput at very high registration volumes.

Why we use JWT & bcrypt: All private API endpoints in BABYNUTI require a valid JWT Bearer token, which is verified on every request by the authentication middleware. User passwords are hashed with bcrypt before storage, ensuring that plaintext credentials are never persisted in the database.

2.13. Multer

Multer is a Node.js middleware library for handling multipart/form-data requests, which is the encoding type used for HTML file uploads. It processes incoming binary file data and makes uploaded files accessible to route handlers.

Advantages of multer:

- Handles binary file data cleanly without requiring manual stream parsing.
- Supports configurable storage destinations, file size limits, and file type filtering.
- Integrates seamlessly into Express.js middleware chains.

Disadvantages of multer:

- Requires explicit configuration to restrict acceptable file types and maximum upload sizes to prevent abuse.
- Does not natively support streaming files directly to cloud storage without additional adapters.

Why we use multer: BABYNUTI uses multer on the backend to handle image uploads for recipe assets and user profile avatars, enabling parents and experts to attach photos to their content through the mobile application.

2.14. Google Cloud Translate API

The Google Cloud Translate API is a machine translation service provided by Google that supports over 100 languages. It is integrated into the BABYNUTI backend via the @google-cloud/translate npm package.

Advantages of Google Cloud Translate API:

- Provides high-quality machine translation across a wide range of language pairs.
- Simple REST-based integration with official SDK support for Node.js.
- Supports on-demand translation of arbitrary text content without requiring pre-translated resources.

Disadvantages of Google Cloud Translate API:

- Requires a billable Google Cloud project and API key, introducing an ongoing operational cost.
- Translation quality for domain-specific nutritional terminology may occasionally require human review.

Why we use Google Cloud Translate API: BABYNUTI integrates automatic translation to allow parents to read expert nutrition articles in their preferred language, making the application accessible to a broader multilingual audience without requiring the content team to manually maintain multiple language versions.

2.15. GitHub

GitHub is a cloud-based platform for version control and collaborative software development built on the Git distributed version control system. It provides tools for code hosting, branch management, pull request reviews, and team collaboration.

Advantages of GitHub:

- Enables parallel feature development through Git's branching model, isolating in-progress work from the stable codebase.
- Pull request reviews provide a structured mechanism for code quality assurance before changes are merged.
- Maintains a complete commit history, offering a full audit trail of all project changes.

Disadvantages of GitHub:

- Merge conflicts can arise when multiple developers modify overlapping sections of code, requiring manual resolution.
- Effective use requires team familiarity with Git branching workflows and conventions.

Why we use GitHub: BABYNUTI adopts a feature-branch workflow where each team member develops assigned features on dedicated branches, keeping in-progress work isolated from the stable main branch and ensuring code integrity across the entire development team.

CHAPTER 3. APPLICATION ANALYSIS, DESIGN AND IMPLEMENTATION

3.1. System Description

BABYNUTI is built on a three-tier client-server architecture that cleanly separates the mobile presentation layer, the backend application logic, and the relational data store.

- Presentation Layer: A React Native Android application with TypeScript. The client communicates with the backend over REST/HTTP and manages state through Redux Toolkit and Zustand.
- Application Layer: A Node.js/Express.js RESTful API server. Business logic is organized into route, controller, and service layers with JWT-based authentication middleware.
- Data Layer: A MySQL database accessed via a mysql2 connection pool, containing 28 tables with full referential integrity enforced through foreign key constraints.

3.2. Requirement Analysis

3.2.1. Functional Requirements

The functional scope of BABYNUTI is organized into six core modules:

- Account Management: Users authenticate securely via one-tap Google Sign-In powered by Firebase Authentication. Upon successful Firebase token verification by the Express.js backend, a custom JWT access token is issued and persisted to AsyncStorage to authenticate all subsequent private API calls.
- Child Profile Management: Authenticated parents can create, view, edit, and delete child profiles. A shareable invitation code allows a second caregiver to gain editor access.
- Recipe Discovery: All users can browse, search, and filter the recipe catalog. Authenticated users can rate, comment on, and bookmark recipes.
- Meal Planning: Parents can build a weekly meal plan for a selected child by assigning recipes to specific dates and meal slots.
- Growth Tracking: Parents can record a child's height and weight over time. The system auto-calculates BMI and renders an interactive time-series chart.

- Content & Settings: Expert-authored nutrition articles are accessible to all users. Parents can configure the application language, theme, and preferred measurement units.

3.2.2. Non-Functional Requirements

- Security: All private endpoints require a valid JWT Bearer token. Passwords are hashed with bcrypt.
- Reliability: The meal planning module falls back to locally persisted AsyncStorage data when the backend is unreachable.
- Usability: The application supports multiple display languages and configurable measurement units.

3.2.3. Use Case Diagram

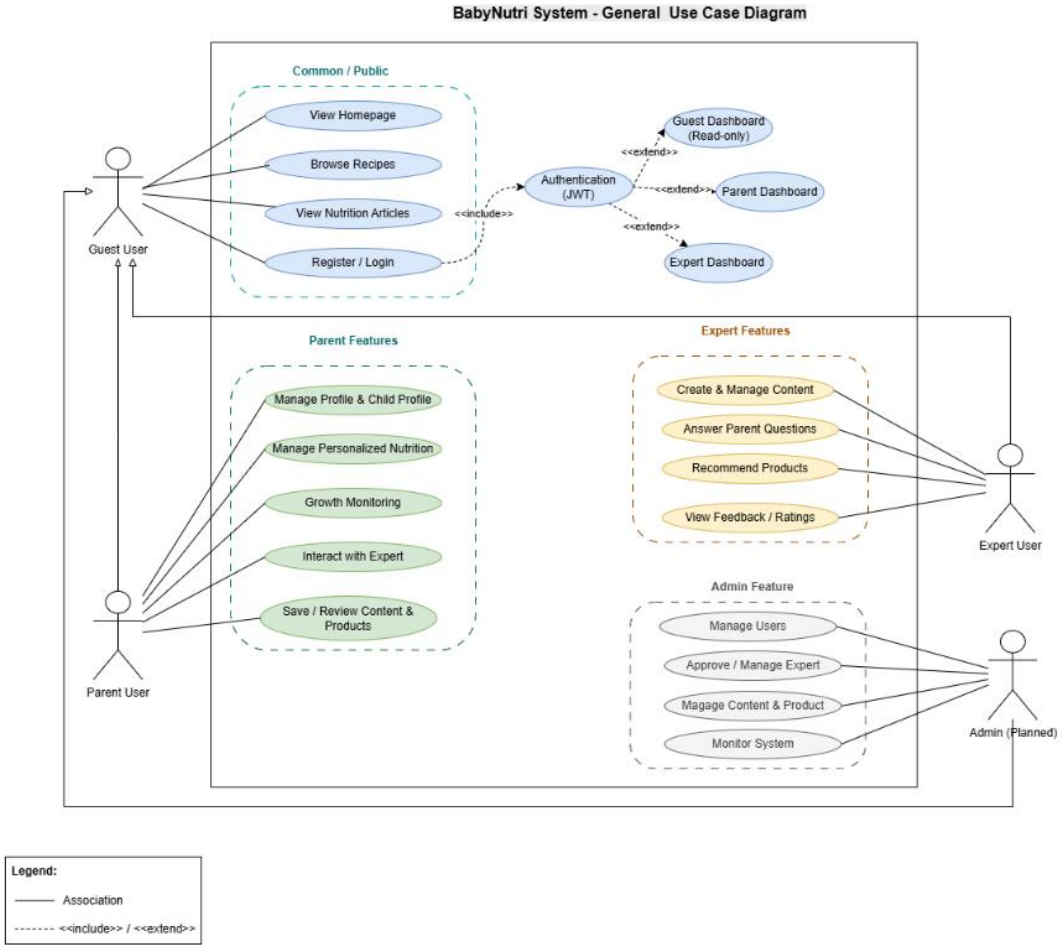

Figure 1. Use Case Diagram

The system defines four primary actors:

- Guest (unauthenticated): Can browse public weaning recipes, explore nutritional articles, and access authentication screens.

- Parent (authenticated user): Inherits all Guest capabilities, manages child profiles and co-caregiver invitations, plans weekly meal schedules, tracks anthropometric growth against WHO standards, and configures application settings (theme, language, units).
- Nutrition Expert: Certified nutritionists who author and publish evidence-based weaning recipes and nutritional articles, provide professional advice via Q&A consultations, and review community feedback.
- System Admin: Platform administrators responsible for managing user accounts, verifying expert medical credentials, moderating user-generated content, and monitoring system health.

3.2.4. Use Case Specification

- Actor: Guest, Parent
- Description: Allows the user to search for weaning recipes matching the child's age and free from specified allergens.
- Precondition: The user is on the Recipe List or Search screen. A network connection is available.
- Main Flow:
    1. The user opens the Search screen, selects a minimum age, and chooses one or more allergy labels.
    2. The client sends a request to the backend.
    3. The backend queries the recipes table and returns a filtered list.
    4. The system displays the matching recipes as cards.
- Alternative Flow: If no recipes match, an empty state is displayed with a suggestion to adjust the filters.
- Postcondition: A filtered list of age-appropriate, allergen-safe recipes is displayed.

3.3. System Architecture

The BABYNUTI system is designed following a three-tier client-server architecture, separating the mobile client, the backend API, and the database layer. This separation of concerns ensures that business logic and data persistence remain independent from the presentation layer, allowing each component to be developed, tested, and scaled independently.

- Presentation layer (mobile client): Built with React Native and TypeScript, this layer is responsible for rendering the user interface, managing local UI state, and handling user interactions such as managing child profiles, discovering recipes, planning weekly meals, and viewing growth tracking charts.
- Application layer (backend API): Built with Express.js, this layer exposes a RESTful API that handles authentication, business logic, input validation, and communication with the database. JWT-based middleware protects private routes, ensuring that only authenticated parents can access or modify their data.
- Data layer (database): MySQL serves as the relational database, accessed through a mysql2 connection pool, which manages relational data persistence and maintains foreign key constraints between application entities**.**

3.4. Activity Diagrams

3.4.1. User Authentication Activity Diagram

The user logs in via email or Google. The client sends an authentication request to the backend. If valid, a JWT token is generated, saved locally, and the user is directed to the HomeScreen; otherwise, an error is displayed.

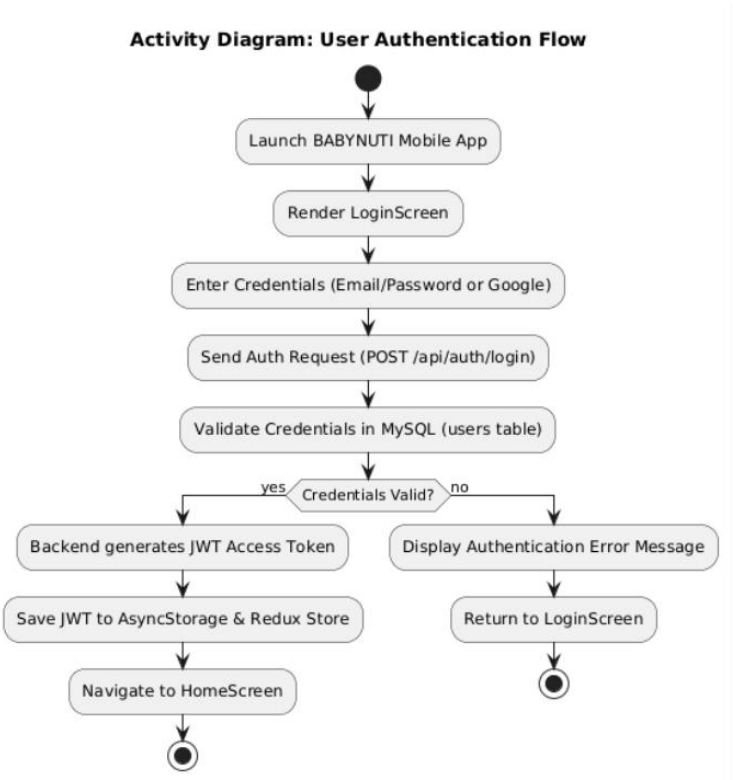

Figure 2. User Authentication Activity Diagram

3.4.2. Child Profile Creation Activity Diagram

The parent inputs child details and submits the form. After client-side validation, the backend processes the request. Valid data is saved to the MySQL database, triggering a UI refresh, while invalid inputs prompt correction alerts.

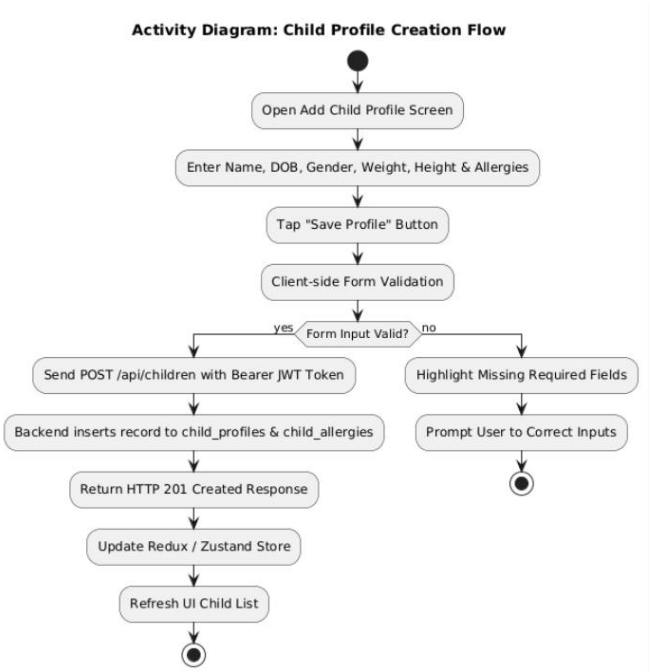

Figure 3. Child Profile Creation Activity Diagram

3.4.3. Recipe Search & Filtering Activity Diagram

The user applies search keywords and filters, such as age and allergies. The backend executes a database query based on these criteria. Matching recipes are displayed as cards, or an empty state is shown if no results are found.

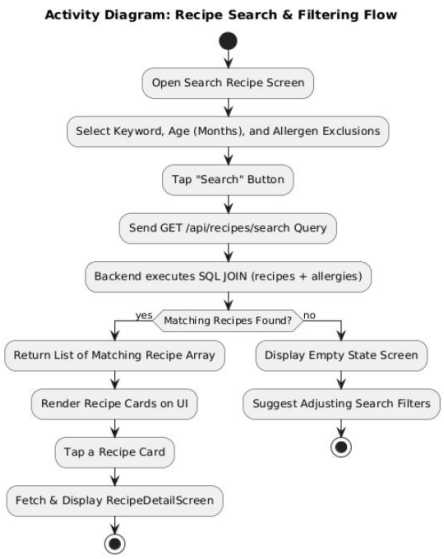

Figure 4. Recipe Search & Filtering Activity Diagram

3.4.4. Growth Tracking & WHO Chart Activity Diagram

The parent inputs new height and weight measurements. The app calculates the BMI and sends the data to the backend for storage and WHO status classification. The saved record is then returned to instantly update the interactive growth chart.

Figure 5. Growth Tracking Activity Diagram

3.4.5. Expert Q&A Consultation Activity Diagram

An expert selects a parent's question and submits an answer. The backend verifies the expert's role permissions and saves the response. The question status is updated to resolved, and a notification is sent to the parent.

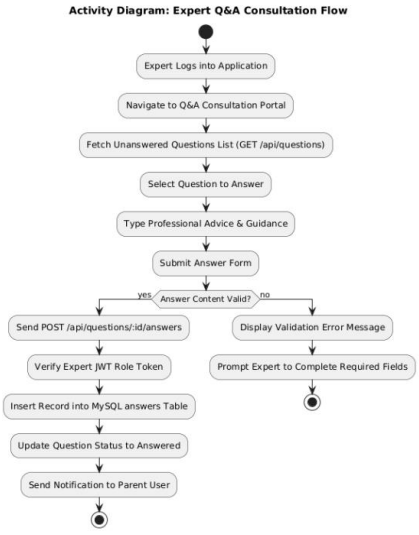

Figure 6. Expert Q&A Consultation Activity Diagram

3.4.6. Admin Recipe & Content Management Activity Diagram

An admin submits data to create or edit content (recipes/articles). The backend verifies the admin role and input validity. Valid requests update the MySQL database and immediately publish the content to the mobile app.

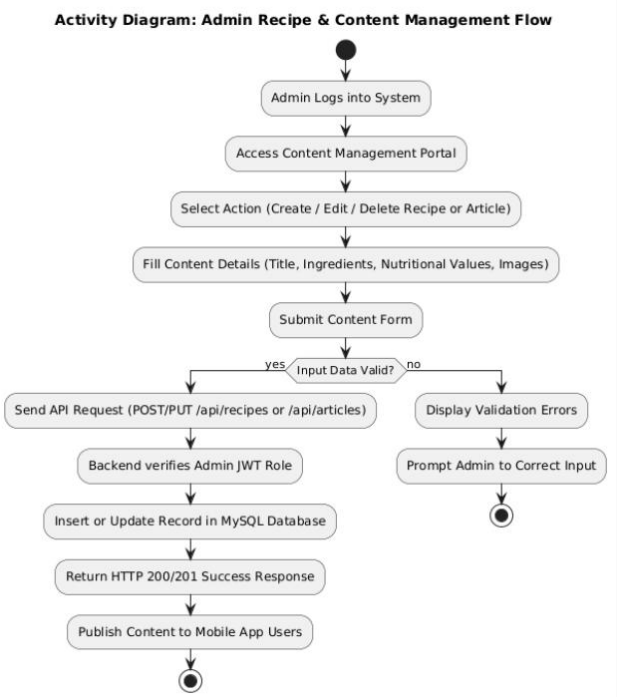

Figure 7. Admin Recipe & Content Management Activity Diagram

3.5. System Design

3.5.1. Database Design

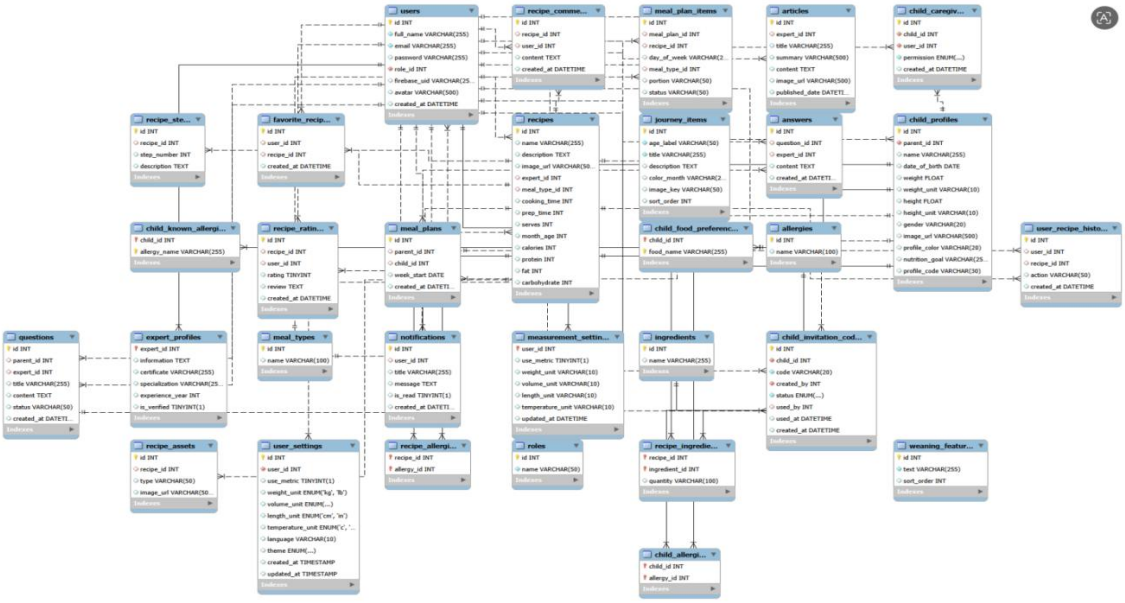

Figure 8. Entity Relationship Diagram (ERD) of BabyNutri Database

a) **Roles:** Stores user role types in the system

Table 1. Roles Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each role.
2	name	VARCHAR(50), NOT NULL, UNIQUE	Role name, for example: Admin, Expert, or Parent.

b) **Users:** Stores account information for all system users.

Table 2. Users Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each user account.
2	full_name	VARCHAR(255), NOT NULL	Full name of the user.
3	email	VARCHAR(255), NOT NULL, UNIQUE	Email address used for account identification and login.
4	password	VARCHAR(255), NOT NULL	User password value stored by the system.
5	role_id	INT, NOT NULL, FK -> roles(id)	Identifies the role assigned to the user.

c) **Child_profiles:** Stores health and basic information of children managed by parent users.

Table 3. Child Profiles Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each child profile.
2	parent_id	INT, NOT NULL, FK -> users(id)	Parent user who owns and manages this child profile.
3	name	VARCHAR(255)	Name of the child.
4	date_of_birth	DATE	Date of birth of the child.
5	weight	FLOAT	Current weight of the child.
6	height	FLOAT	Current height of the child.
7	gender	VARCHAR(20)	Gender of the child.
8	image_url	VARCHAR(500)	Path or URL of the child profile image.

d) **Child Growth Records:** Stores personal profiles and dietary requirements of children managed by parents.

Table 4. Child Growth Records Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each growth record entry.
2	child_id	INT, NOT NULL, FK -> child_profiles(id)	Identifies the child profile associated with the record.
3	record_date	DATE, NOT NULL	The date on which the measurement was taken.
4	weight	FLOAT, NOT NULL	Measured body weight of the child.
5	height	FLOAT, NOT NULL	Measured body height of the child.
6	head_circumference	FLOAT, NULL	Optional measured head circumference.
7	bmi	FLOAT, NOT NULL	Calculated Body Mass Index based on height and weight.
8	status	VARCHAR(50), NULL, DEFAULT 'Healthy Growth'	WHO growth status classification.
9	notes	TEXT, NULL	Additional clinical or parent observation notes.
10	created_at	TIMESTAMP, NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when the measurement record was logged.

e) **Recipes:** Stores recipe information published by nutrition experts.

Table 5. Recipes Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each recipe.
2	name	VARCHAR(255)	Recipe title.
3	description	TEXT	Short description or introduction of the recipe.
4	image_url	VARCHAR(500)	Main image path or URL of the recipe.
5	expert_id	INT, FK -> users(id)	Expert user who creates or publishes the recipe.
6	meal_type_id	INT, FK -> meal_types(id)	Meal category assigned to the recipe.
7	cooking_time	INT	Cooking duration, normally recorded in minutes.
8	prep_time	INT	Preparation duration, normally recorded in minutes.
9	serves	INT	Number of servings produced by the recipe.
10	month_age	INT	Recommended minimum child age in months.

f) Allergies Table : Stores common food allergen classifications to protect children from allergic reactions.

Table 6. Allergies Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each allergen entity.
2	name	VARCHAR(255) , NOT NULL	Name of the food allergen (e.g., Seafood, Dairy, Peanut, Egg, Soy).

g) Meal_plans: Stores weekly meal plans created by parents for a selected child.

Table 7: Meal Plans Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each meal plan.
2	parent_id	INT, FK -> users(id)	Parent who creates and manages the meal plan.
3	child_id	INT, FK -> child_profiles(id)	Child for whom the meal plan is prepared.
4	week_start	DATE	Starting date of the meal-plan week.

h) meal_plan_items: Stores individual scheduled recipe entries within a meal plan.

Table 8. Meal Plan Items Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each meal-plan item.
2	meal_plan_id	INT, FK -> meal_plans(id)	Meal plan containing this schedule item.
3	recipe_id	INT, FK -> recipes(id)	Recipe scheduled for the specified day and meal.
4	day_of_week	VARCHAR(20)	Day of the week for the scheduled meal.
5	meal_type_id	INT, FK -> meal_types(id)	Meal category for the schedule item.
6	portion	VARCHAR(50)	Recommended portion size for the child.
7	status	VARCHAR(50), DEFAULT 'planned'	Progress status of the planned meal, for example planned or completed.

i) Articles: Stores nutrition articles written or published by experts.

Table 9. Articles Table

No.	Field	Data type / constraints	Description

	name		
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each article.
2	expert_id	INT, FK -> users(id)	Expert author or publisher of the article.
3	title	VARCHAR(255)	Article title.
4	content	TEXT	Main article content.
5	image_url	VARCHAR(500)	Path or URL of the article cover image.

j) Ingredients : Stores a reusable list of ingredients used in recipes.

Table 10. Ingredients Table

No.	Field name	Data type / constraints	Description
1	id	INT, PK, AUTO_INCREMENT	Unique identifier of each ingredient.
2	name	VARCHAR(255)	Ingredient name.

3.5.2. API Design

Table 11. API Endpoint Summary Table

Method	Endpoint	Description
POST	/auth/register, /auth/login, /auth/firebase-login	Register and authenticate a parent user via password or Google Sign-In, issuing a JWT token.
GET / PUT	/parent/avatar, /parent/baby/:id/avatar	Update the profile avatar for the authenticated parent or child profile.
GET / POST / PUT / DELETE	/children, /children/:id	Full CRUD on child profiles including name, date of birth, gender, allergies, and food preferences.
POST	/children/:childId/invitations, /invitations/activate	Generate and activate shareable invitation codes for shared multi-caregiver access.
GET / POST / PUT / DELETE	/children/:childId/growth, /children/:childId/growth/:recordId	Track, update, and retrieve historical height, weight, and BMI growth records with WHO reference standards.
GET / POST / PUT / DELETE	/recipes, /recipes/search, /recipes/:id	Search, filter by age/allergy, view details, and manage weaning recipes.
POST	/recipes/:id/rating, /recipes/:id/comments	Submit star ratings, written reviews, and comments on recipes.
GET / POST	/mealplans, /mealplans/:id	Create, retrieve, and update weekly meal plans for a child.
GET / POST / PUT /	/articles, /articles/:id	Browse, read, translate, and manage expert nutrition articles.

DELETE		
GET / PUT	/measurement-settings, /translate, /home	Manage measurement unit preferences (kg/lb, cm/in), translate article content, and fetch homepage journey items.and fetch homepage journey items.

3.6. Implementation

3.6.1. Backend Implementation

- The backend source is organized into four main directories:
- routes/: Maps URL paths to their corresponding controller functions, organizing API endpoints logically by resource.
- controllers/: Parses incoming HTTP requests, validates inputs, and returns formatted JSON responses, keeping the core business logic isolated from HTTP routing.
- middleware/: Handles request interception, securely verifying custom JWTs and Firebase ID tokens before granting access to protected routes.
- config/: Manages the initialization and configuration of external services, such as the Firebase Admin SDK.
- Database Module: Centralizes database access by exporting a connection pool, which is specifically configured to prevent timezone serialization issues across different environments.

3.6.2. Frontend Implementation

The mobile frontend is structured with the following key modules:

- navigation/: Defines the application's screen hierarchy, utilizing a root stack navigator for the top-level flow and a bottom tab navigator for primary feature accessibility.
- screens/: Organizes all application screens into functional subdirectories, ensuring each UI component remains self-contained.
- services/: Provides a robust HTTP abstraction layer. It automatically attaches authentication tokens to all outgoing requests and implements local-first data patterns to support offline capabilities, particularly for the meal planning module.
- store/: Integrates two state management strategies: Redux Toolkit manages application-wide global state (e.g., authentication, active profile), while Zustand handles lightweight, feature-local state to enable clean optimistic UI updates.

- i18n/: Manages localization and translation resources, allowing the application to dynamically update its display language based on user preferences.

3.7. Summary

This chapter has described the analysis, design, and implementation of the BABYNUTI mobile application. The system employs a three-tier architecture with a React Native frontend, a Node.js/Express.js RESTful API, and a MySQL database. Six functional modules are supported by 28 database tables and over 30 API endpoints. The frontend architecture combines React Navigation for routing, Redux Toolkit and Zustand for state management, and a typed service layer for backend communication.

CHAPTER 4. RESULT

This chapter presents the implemented user interface and user experience of the BABYNUTI mobile application. It describes the visual layout, screen interactions, and functional navigation flows across all major application modules as experienced by end users on an Android device.

4.1. Authentication Flow

When users launch the application for the first time, they are welcomed by the Welcome screen. This screen features the application logo, a welcoming slogan, and two prominent action buttons: *Log In* and *Get Started*.

Tapping *Get Started* navigates to the Register screen, where new users can create an account by providing their full name, email address, and password. Tapping *Log In* opens the Login screen, which offers a standard email/password form alongside a *Sign in with Google* option. Tapping the Google sign-in button opens the native Google account selection dialog. Once authenticated through either method, the system automatically redirects the user from the authentication screens to the main application interface, displaying the home dashboard.

Figure 9. Welcome Screen Interface

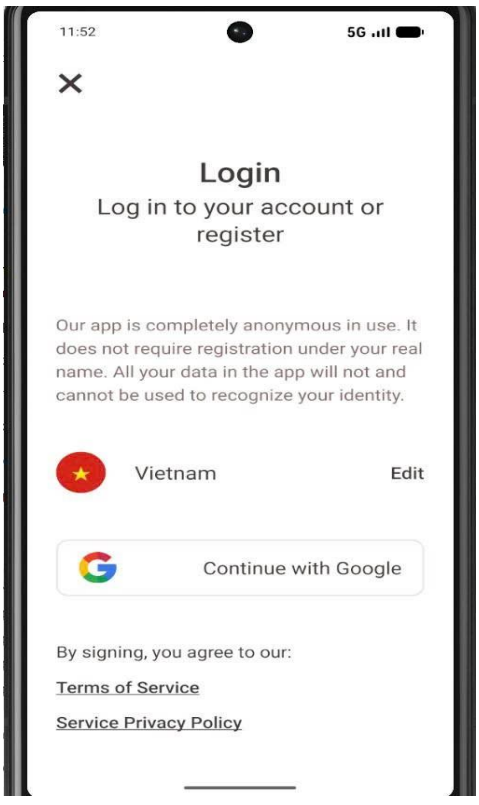

Figure 10. User Login Screen Interface

4.2. Onboarding and Initial Setup

Upon completing initial registration, parents are guided through a structured 9-step onboarding questionnaire designed to establish the child's physical baseline and personalize dietary recommendations. The sequence begins with an introductory feature overview highlighting core application capabilities.

Parents then proceed through an intuitive multi-step setup form to input the child's identity details - including name, date of birth (used for monthly age calculation), gender, and baseline growth metrics (weight in kg/lbs and height in cm/inches). In the final stages of the onboarding wizard, the system surveys the child's known food allergies (such as dairy, eggs, soy, and shellfish), primary nutritional goals (e.g., healthy growth, balanced diet), and food taste preferences (grains, vegetables, meat, fish). Upon completion, this profile data is persisted to the MySQL database, and the user is immediately routed to the personalized Home Dashboard.

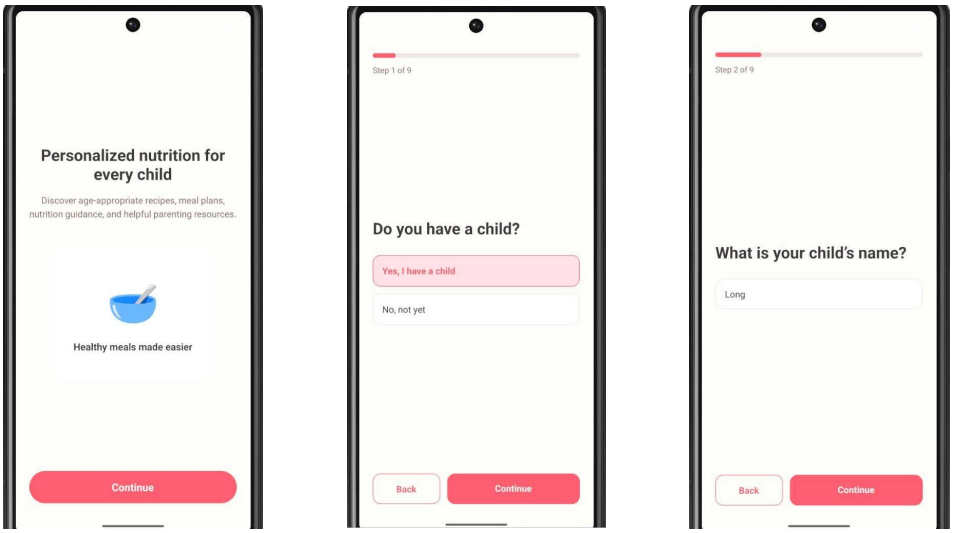

Figure 11. Onboarding Wizard — Introduction, Child Confirmation, and Name Setup (Steps 1–2)

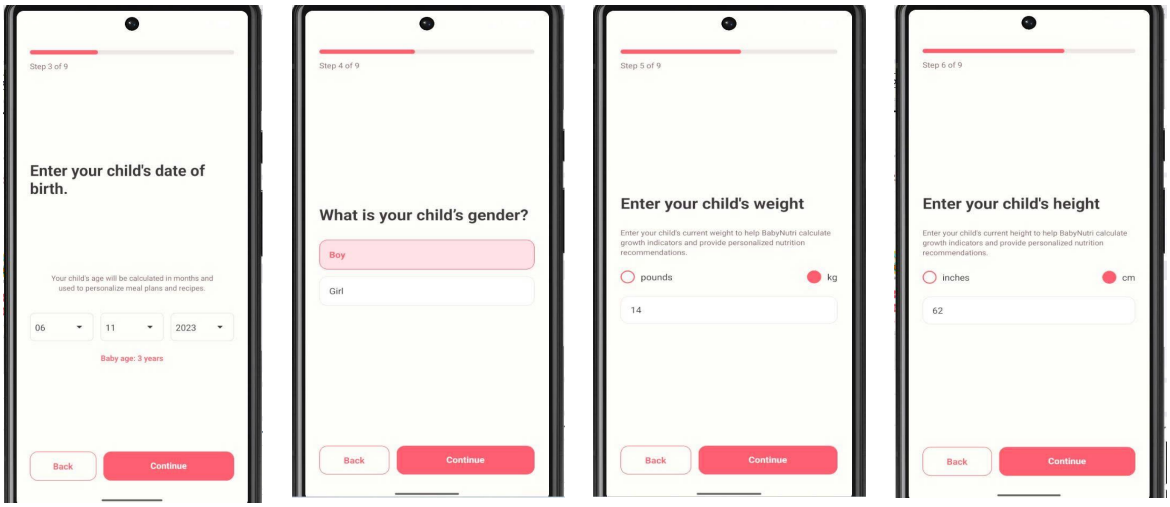

Figure 12. Onboarding Wizard — Date of Birth, Gender, and Baseline Anthropometric Measurements (Steps 3–6)

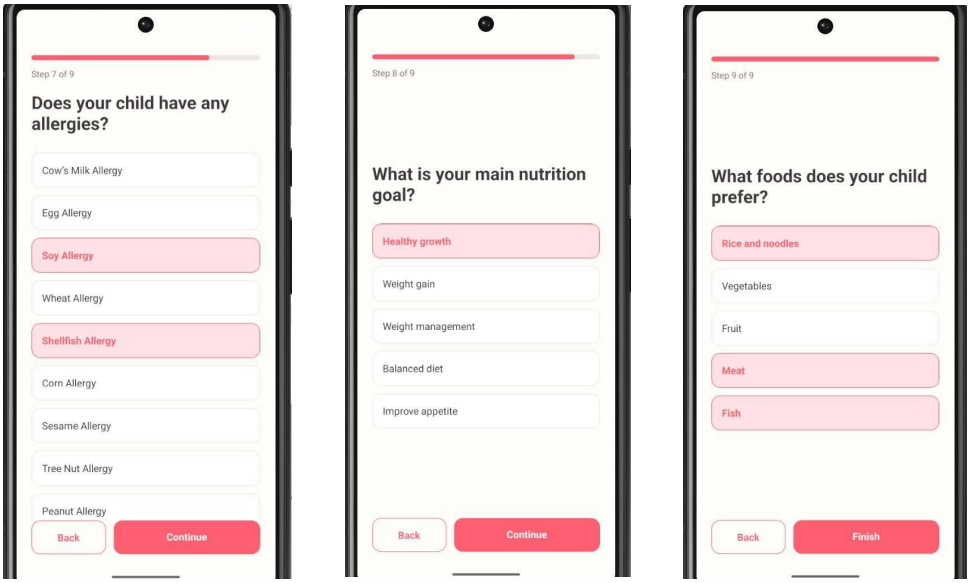

Figure 13. Onboarding Wizard — Known Allergen Screen, Nutrition Goals, and Food Preferences Survey (Steps 7–9

4.3. Home Dashboard

The Home screen serves as the primary dashboard of the application. At the top of the interface, a header card displays the parent's greeting alongside the active child's avatar, name, and current age.

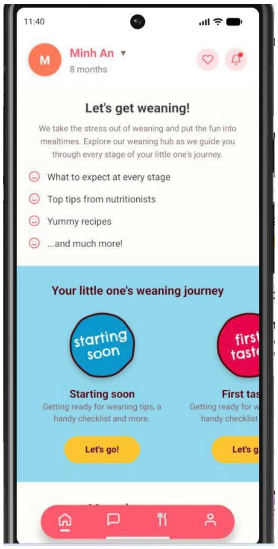

Figure 14. Main Home Dashboard and Weaning Journey Screen

Below the header, a horizontally scrollable Journey section presents age-specific weaning guides (such as *"First Tastes"*, *"From 6 Months"*, and *"From 10 Months"*), each rendered with distinct colors and illustrative icons. Swiping further down reveals a curated

selection of recommended weaning recipes and daily nutrition highlights, allowing parents to quickly access popular content directly from the main tab.

4.4. Recipe Discovery and Management

The Recipe List screen is accessible via the bottom navigation bar and displays weaning recipes as visual cards containing the recipe title, target age range, meal category, and estimated cooking time. A category filter bar at the top of the screen allows users to filter recipes by meal types, including *Breakfast*, *Lunch*, *Dinner*, and *Snack*.

Tapping the search bar transitions to the Search Recipe screen, where users can enter keywords, filter recipes by age group categories (*All*, *6–12 months*, *12–24 months*, *24+ months*), and tap the Search Recipes button. The displayed recipe cards update dynamically from the backend API, presenting suitable age tags, calorie counts, and preparation times.

Selecting a recipe card opens the Recipe Detail screen, which presents comprehensive cooking and nutritional guidance. The upper portion displays a high-resolution dish image, prep time, cooking time, and a macronutrient summary showing total calories, protein, fat, and carbohydrates. The main body lists required ingredients with exact quantities and step-by-step preparation instructions. At the bottom of the screen, users can view average star ratings and community reviews, submit their own 1–5 star ratings and comments, or tap the bookmark icon in the header to save the recipe for future reference.

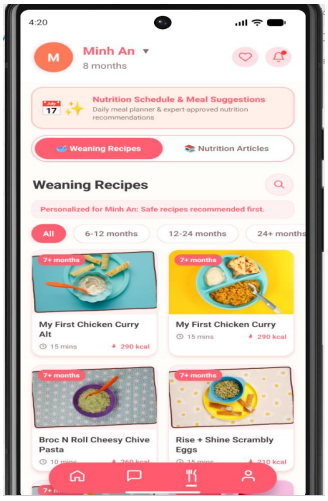

Figure 15. Weaning Recipe Catalog List Screen

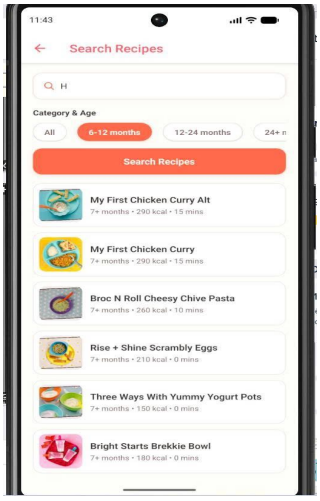

Figure 16. Recipe Search and Age Filter Screen

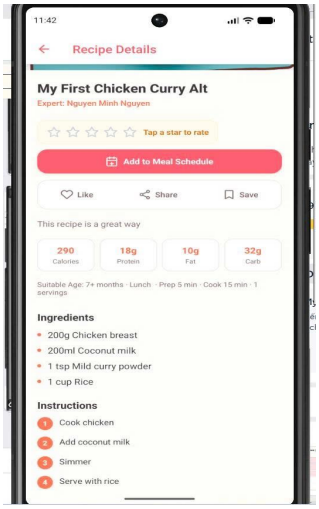

Figure 17. Recipe Detailed Nutritional Breakdown and Instructions Screen

4.5. Meal Planning Module

The meal planning module assists parents in organizing their child's weekly diet. The Meal Plan List screen displays weekly schedules organized into daily cards, summarizing

total daily calorie and macronutrient goals. Tapping a specific day opens the Meal Plan Detail screen, which organizes scheduled dishes into distinct meal slots (*Breakfast*, *Lunch*, *Snack*, and *Dinner*), showing individual nutritional figures alongside total daily progress bars.

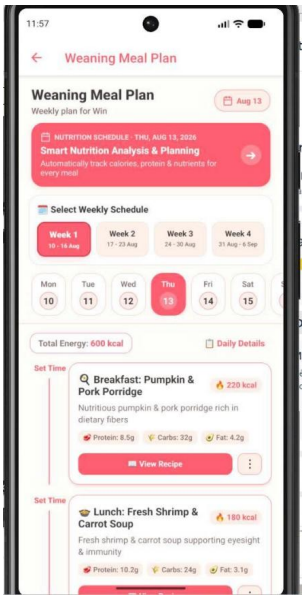

Figure 18. Weaning Meal Plan and Daily Detail Screen

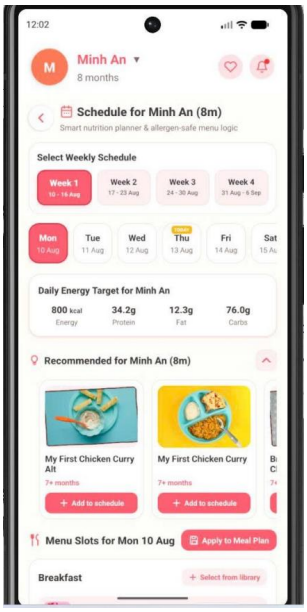

Figure 19. Smart Meal Scheduler and Nutrition Planner Screen

Parents can add dishes directly from the Recipe Detail screen by selecting the *Add to Meal Plan* option. This opens a modal selection sheet where the target date and meal slot are specified. If a dish has already been assigned to the same child on that date, the application presents a confirmation message to prevent duplicate scheduling.

The Meal Scheduler screen provides an interactive weekly calendar row. Selecting a date from the calendar updates the schedule view below, displaying all assigned meals for that day in a clean, chronological timeline.

4.6. Child Profile and Growth Tracking

The Child List screen, accessible from the profile tab, lists all child profiles associated with the user account. Tapping a profile opens the Child Detail screen, presenting the child's personal details, age, avatar, known allergies, and food preferences, along with quick action buttons for growth tracking, editing profile information, and sharing access with other caregivers.

The Growth Tracking screen features an interactive growth chart that plots the child's recorded height and weight history against official World Health Organization (WHO) standard percentile curves. Beneath the chart, historical records are listed in reverse chronological order, displaying the measurement date, height, weight, calculated Body Mass

Index (BMI), and growth classification status. Tapping the *Add Record* button opens an input modal where parents can enter new measurements, dates, and optional notes. Once submitted, the growth chart refreshes immediately to reflect the new data point.

To share a child's profile with another caregiver, parents can generate an invitation code from the Child Detail screen. The secondary caregiver can then navigate to the Invitation Code screen on their own device, enter the code, and instantly gain shared editing access to the child's profile and growth records.

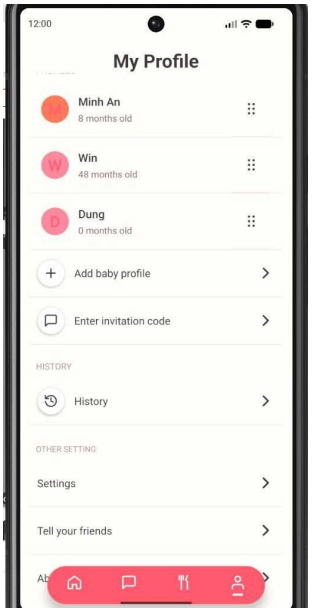

Figure 20. Child Profiles Management List Screen

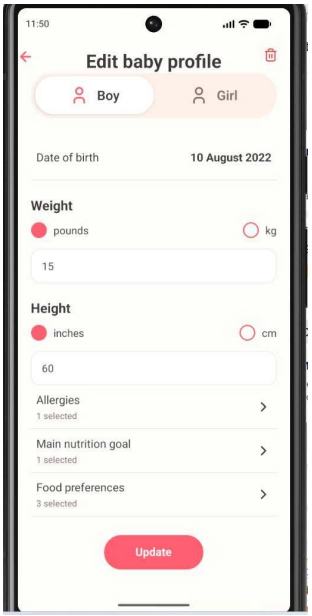

Figure 21. Child Profile Detail and Anthropometric Editor Screen

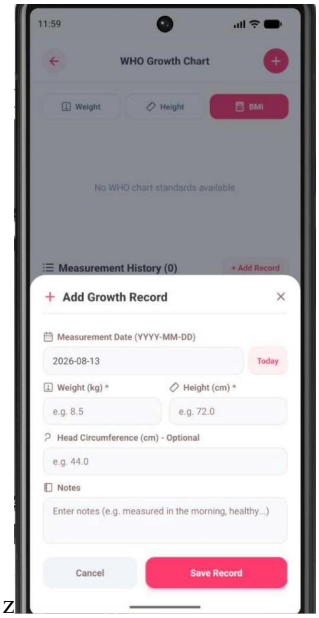

Figure 22. WHO Growth Tracking and Anthropometric Measurement Logging Screen

4.7. Articles, Community

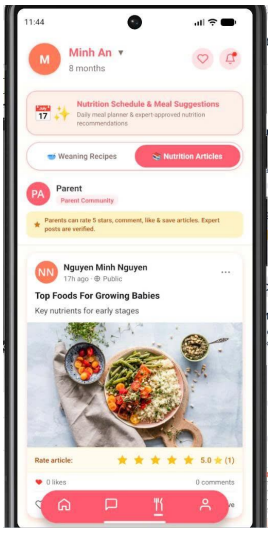

Figure 24. Article Details and Community Discussion

Screen

Figure 23. Nutrition Articles and Library Feed Screen

The Article List screen presents expert-written educational content organized into visual cards featuring titles, summary excerpts, author details, and cover images. Tapping an article opens the Article Detail screen, displaying the full text alongside an *Auto-Translate* button that allows users to view the content in their preferred language. Parents can also share their own insights by creating community posts via the Add Article screen.

4.8. Administrative Governance and Expert Content Management

- Nutritionist Workspace: Provides a dedicated dashboard to manage published recipes, educational articles, and track cumulative parent satisfaction ratings.
- Recipe Creation Interface: Enables specialists to publish infant meals with target developmental stages, macronutrient metrics, allergen disclosures, dynamic ingredients, and step-by-step preparation guides.
- Administrative Center: Offers real-time platform governance to monitor overall user/expert growth, content catalogs, and community Q&A engagement.

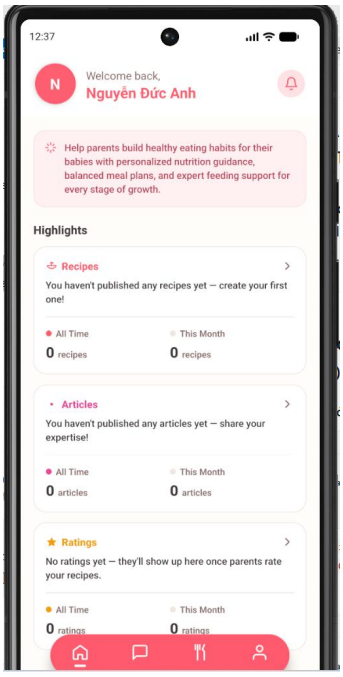

Figure 25. Expert Content Management and Highlights Analytics Dashboard

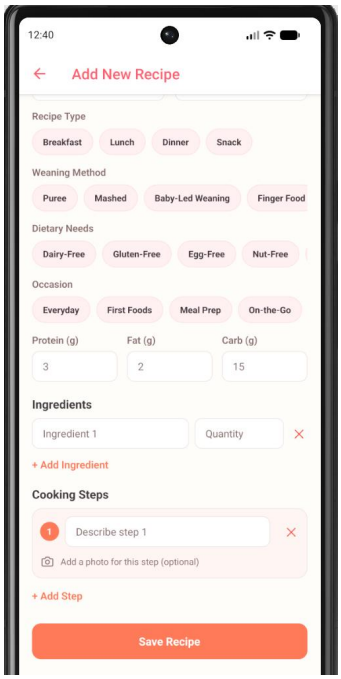

Figure 26. Expert Weaning Recipe Authoring and Nutritional Breakdown Screen

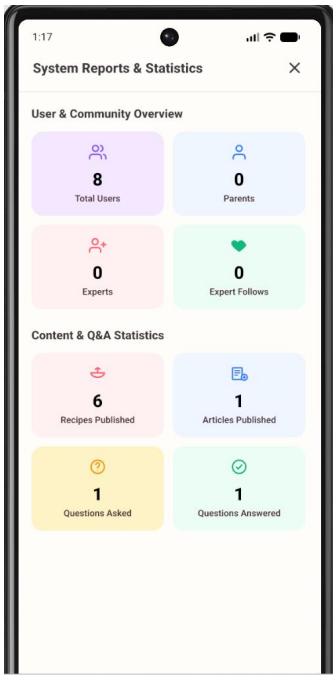

Figure 27. Administrator System Analytics and Statistical Overview Screen

CHAPTER 5. CONCLUSION AND FUTURE WORKS

5.1. Conclusion

This project has successfully delivered a functional Android mobile application — BABYNUTI - addressing the core challenge of providing parents with a unified, structured platform for managing infant nutrition and monitoring child development during the weaning stage. The application is built on a modern full-stack JavaScript architecture comprising React Native (v0.86.0) with TypeScript for the mobile client, Node.js and Express.js (v5) for the RESTful API backend, and MySQL for relational data persistence.

Across its 37 implemented screens and 30+ API endpoints, the application delivers six functional modules: dual-mode authentication (email/password and Google Sign-In via Firebase), child profile management with multi-caregiver support, recipe discovery with age and allergy filtering, weekly meal planning with a local-first offline strategy, growth tracking with WHO percentile chart visualization, and multilingual nutrition content with automatic translation. User preferences including display language, color theme, and measurement units are persisted across sessions via AsyncStorage and Redux.

The project demonstrates practical application of several architectural patterns encountered in professional mobile development: the separation of global and feature-local state through the combined use of Redux Toolkit and Zustand, optimistic UI updates for responsive interactions, graceful degradation to locally cached data under poor network conditions, and a dual authentication middleware strategy that separates the Google identity layer from the application's own authorization logic.

Key limitations of the current version include the absence of a native iOS build, no automated test suite, and the lack of push notification support for meal reminders and growth milestones. The meal planning module's backend synchronization is also one-directional in the current implementation, with full bidirectional sync deferred to future development.

5.2. Future Works

Several features identified in the original product requirements document were out of scope for this academic iteration of the project. The following enhancements are proposed for subsequent development cycles:

1. iOS Platform Support. The application is currently built and tested exclusively for Android. Extending the React Native codebase to support iOS would require platform-

specific native configuration (Xcode, Apple Sign-In) but no fundamental architectural changes, as the shared TypeScript codebase is designed for cross-platform compatibility.

2. AI-Powered Recipe Recommendations. The initial requirements described a personalized recommendation engine that surfaces recipes based on the child's age, allergy profile, growth status, and meal history. Implementing this feature would involve training or integrating a lightweight collaborative filtering model, backed by the user_recipe_history table already present in the database schema.
3. Push Notifications. The current application provides no proactive reminders. A future release should integrate Firebase Cloud Messaging (FCM) to send scheduled notifications for meal times, growth measurement reminders, and new expert article publications features explicitly requested in the original product brief.
4. Real-Time Expert Q&A Chat. The original requirements included a direct messaging feature connecting parents with verified nutrition experts.
5. The questions and answers tables provide a foundation for an asynchronous Q&A system, but a real-time implementation would require integrating a WebSocket layer (e.g., Socket.io) or a managed messaging service such as Firebase Realtime Database.
6. Offline-First Synchronization. The current offline strategy is limited to meal plan data cached in AsyncStorage. A comprehensive offline-first architecture covering child profiles, growth records, and recipe bookmarks with conflict-resolving background sync would significantly improve reliability in environments with intermittent connectivity.
7. Automated Testing and CI/CD Pipeline. The current codebase contains no unit, integration, or end-to-end tests. Future development should introduce Jest unit tests for service functions and Redux slices, Detox end-to-end tests for critical user flows, and a GitHub Actions CI/CD pipeline to automate testing and build validation on each pull request.
8. E-Commerce Integration. The original product vision included the ability for parents to purchase nutrition-related products such as age-appropriate food items and weaning equipment directly within the application. This feature would require integration with a payment gateway (e.g., Stripe) and a product catalog API, both of which are currently absent from the system.

REFERENCES

[1] World Health Organization (WHO), "WHO Child Growth Standards: Length/height-for-age, weight-for-age, weight-for-length, weight-for-height and body mass index-for-age: Methods and development," WHO Press, Geneva, Switzerland, Tech. Rep., 2006. [Online]. Available: https://www.who.int/tools/child-growth-standards

[2] World Health Organization (WHO), "Infant and Young Child Feeding: Model Chapter for Textbooks for Medical Students and Allied Health Professionals," World Health Organization, Geneva, Switzerland, 2009. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK148965/

[3] M. Fewtrell, J. Bronsky, C. Campoy, M. Domellöf, N. Embleton, N. Fidler Mis, I. Hojsak, J. M. Hulst, F. Indrio, A. Lapillonne, and C. Molgaard, "Complementary Feeding: A Position Paper by the European Society for Paediatric Gastroenterology, Hepatology, and Nutrition (ESPGHAN) Committee on Nutrition," Journal of Pediatric Gastroenterology and Nutrition, vol. 64, no. 1, pp. 119–132, Jan. 2017. doi: 10.1097/MPG.0000000000001454.

[4] R. E. Kleinman and F. R. Greer, Eds., Pediatric Nutrition, 8th ed. Elk Grove Village, IL, USA: American Academy of Pediatrics (AAP), 2019.

[5] Meta Platforms, Inc., "React Native: A framework for building native apps using React," 2026. [Online]. Available: https://reactnative.dev/

[6] Microsoft Corporation, "TypeScript: JavaScript With Syntax For Types," 2026. [Online]. Available: https://www.typescriptlang.org/

[7] React Navigation Contributors, "React Navigation: Routing and navigation for Expo and React Native apps," 2026. [Online]. Available: https://reactnavigation.org/

[8] M. Abramov, D. Clark, and Redux Contributors, "Redux Toolkit: The official, opinionated, batteries-included toolset for efficient Redux development," 2026. [Online]. Available: https://redux-toolkit.js.org/

[9] D. Kato and Poimandres, "Zustand: A small, fast and scalable bearbones state-management solution using simplified flux principles," 2026. [Online]. Available: https://github.com/pmndrs/zustand

[10] M. Zabriskie and Axios Contributors, "Axios: Promise based HTTP client for the browser and node.js," 2026. [Online]. Available: https://axios-http.com/

[11] Google LLC, "Firebase Authentication: Simple, multi-platform sign-in," Google Cloud, 2026. [Online]. Available: https://firebase.google.com/docs/auth

[12] React Native Community, "AsyncStorage: An asynchronous, unencrypted, persistent, key-value storage system for React Native," 2026. [Online]. Available: https://react-native-async-storage.github.io/async-storage/

[13] J. Mühlemann and i18next Contributors, "i18next: Internationalization-framework written in and for JavaScript," 2026. [Online]. Available: https://www.i18next.com/

[14] OpenJS Foundation, "Node.js: JavaScript runtime built on Chrome's V8 JavaScript engine," 2026. [Online]. Available: https://nodejs.org/

[15] StrongLoop and Express.js Contributors, "Express: Fast, unopinionated, minimalist web framework for Node.js," 2026. [Online]. Available: https://expressjs.com/

[16] Oracle Corporation, "MySQL 8.0 Reference Manual," Oracle Corporation, Redwood Shores, CA, USA, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[17] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force (IETF), RFC 7519, May 2015. doi: 10.17487/RFC7519.

[18] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proceedings of the USENIX Annual Technical Conference, FREENIX Track, Monterey, CA, USA, Jun. 1999, pp. 81–91. [

19] Express.js Contributors, "Multer: Node.js middleware for handling multipart/form-data," 2026. [Online]. Available: https://github.com/expressjs/multer

[20] Google LLC, "Cloud Translation API: Neural Machine Translation across languages," Google Cloud Documentation, 2026. [Online]. Available: https://cloud.google.com/translate

[21] G. Rauch and Socket.IO Contributors, "Socket.IO: Bidirectional and low-latency communication for every platform," 2026. [Online]. Available: https://socket.io/

[22] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.